Push notifications: step-by-step build guide

Oct 30, 2026 · @danial

Overview

By the end of this guide, one logged-in user can send a push notification to another user's phone, using a bell in the Form Depot header and a Send button on a new Notifications page. Follow the parts in order: A, B, C, D, then E to test.

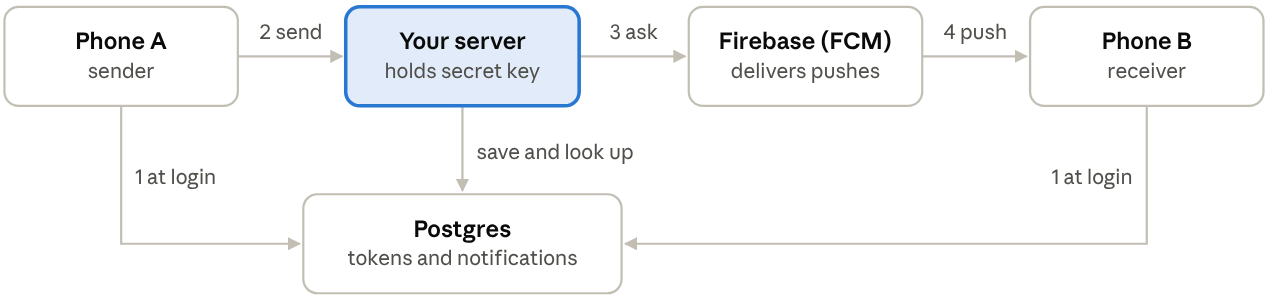
What you will have built

- A bell with a badge in the Form Depot tab header. Tapping it opens a Notifications page.
- On that page, a "Send to username" box and a **Send notification** button.
- At login, each phone tells your server "this token belongs to this user". At logout, it removes that link.
- A small server (Express + Postgres) that stores the tokens and the notifications, and asks Firebase to deliver the push.

How a notification travels (phone A sends to phone B)

1. Both phones log in. Each one gets its Firebase token and sends it to your server, which saves *username → token* in the database.
2. On phone A you type B's username and tap **Send notification**. The app calls your server.
3. The server saves the notification, finds B's token(s) in the database, and asks Firebase to deliver.
4. Firebase delivers the push to phone B. If a token is dead (for example the app was reinstalled), the server deletes it.

Your server, not the app, asks Firebase to deliver the push



The app never sends through Firebase itself. Only your server holds the Firebase secret key.

push flow · 4 steps, your server in the middle

The numbers match the list above: 1 is the login step (both phones), and 2 to 4 are the send.

The app never talks to Firebase to send. Only your server holds the Firebase secret key.

Files that change

File	Change	Where in this guide
`lib/firebase_options.dart`	New, generated by a tool	Part A, step A3
`lib/constant/fcm_config.dart`	New	Part D, step 1
`lib/noti_services/push_service.dart`	New	Part D, step 2
`lib/screen/notification/notification_screen.dart`	New	Part D, step 7
`lib/main.dart`	Edited	Part D, step 3
`lib/controller/auth/auth_controller.dart`	Edited	Part D, step 4
`lib/controller/home/root_home_controller.dart`	Edited	Part D, step 5
`lib/constant/routes.dart`	Edited	Part D, step 6
`lib/screen/home/root_home_screen.dart`	Edited	Part D, step 8

Nothing else inside `lib` is touched. Outside `lib` you also change `pubspec.yaml`, two Android files, and create a database and a small server folder.

Before you start: you need Flutter working, Node.js (LTS), Docker or a local Postgres, a Google account, and an Android phone. A second device with a second account is needed only for the final test.

Part A: Setup outside lib

Part A connects your app to Firebase, Google's push service. Run every command in your app's root folder (the one that contains `lib`, `pubspec.yaml` and `android`) unless a step says "anywhere". We use Windows PowerShell.

A1. Create a Firebase project

1. Go to console.firebase.google.com and sign in with a Google account.
2. Click **Create a project**, name it (ours is `trams-test`, project id `trams-test-f2a06`), turn Google Analytics off, and create it.
3. This is a test project. When the app goes to production, your organisation's own Firebase project replaces it: repeat A3 and the Firebase key step in Part C against that project.

A2. Install the tools (anywhere)

Install the LTS version of Node.js from nodejs.org if `node -v` says "not recognized". Then run:

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
```

Close and reopen the terminal after the first command, or `firebase` will not be found. After the third command, Windows warns that `C:\Users\User\AppData\Local\Pub\Cache\bin` is not on your PATH. Add that folder to your user PATH (Windows search, "Edit the system environment variables", Environment Variables, User variables, Path, New), then reopen the terminal. For one window only, this works instead:

```
$env:Path += ";C:\Users\User\AppData\Local\Pub\Cache\bin"
```

A3. Connect the app to Firebase

```
flutterfire configure --project=trams-test-f2a06 --platforms=android
```

Only Android is configured on purpose. Without `--platforms=android` it selects every platform and the web registration fails. This command creates two files: `lib/firebase_options.dart` and `android/app/google-services.json`. It also registers the Android app `com.ai.trams` in your Firebase project. Never type these two files by hand.

A4. Add the packages

```
flutter pub add firebase_core firebase_messaging
```

This adds two lines under `dependencies:` in `pubspec.yaml`. The app already uses `dio` (for network calls), `get` and `get_storage`, so nothing else is needed.

A5. Check the Android build files

Firebase needs the `google-services` plugin in two files. Check that both have it:

```
findstr /s /i "google-services" android\*.gradle android\*.kts
```

In our project both already had it:

- `android/settings.gradle`, in the `plugins` block: `id "com.google.gms.google-services" version "4.3.15" apply false`
- `android/app/build.gradle`, in the `plugins` block: `id 'com.google.gms.google-services'`

If either is missing, add that line. If the build later complains about `minSdkVersion`, set it to 23 in `android/app/build.gradle`.

A6. Give Gradle more memory

Our first build with Firebase failed with "Java heap space". Open `android/gradle.properties`, find the `org.gradle.jvmargs=` line (add it if missing), and set:

```
org.gradle.jvmargs=-Xmx4G -XX:MaxMetaspaceSize=2G
```

Use `-Xmx3G` on a PC with 8 GB of RAM. Then stop the old Gradle process so the new value is used:

```
cd android
.\gradlew --stop
cd ..
```

Only if the same heap error still appears, also set `android.enableJetifier=false` in the same file. Warnings that Gradle, the Android Gradle Plugin or Kotlin versions will "soon be dropped" are harmless.

A7. Allow plain `http` on Android (testing only)

In Part E your phone calls the server on your PC at an address like `http://192.168.1.23:4000`. Android blocks plain `http` by default. Open `android/app/src/main/AndroidManifest.xml` and add one attribute to the `<application ...>` tag:

```
<application
    android:usesCleartextTraffic="true"
    ...keep the attributes that are already there...>
```

This is for testing only. A production server uses `https`, and this line is then removed.

Part B: The database

The server needs two tables: one remembers which phone belongs to which user, the other stores the notifications. We use PostgreSQL. If you already have a Postgres you can use, skip B1 and run the SQL from B2 in a database called `trams_noti_db`.

B1. Start a Postgres database

Create a folder for the notification server, for example `fcm_server`, outside your Flutter project. Inside it create `docker-compose.yml`:

```
services:
  noti-db:
    image: postgres:16
    container_name: trams-noti-db
    environment:
      POSTGRES_USER: noti_user
      POSTGRES_PASSWORD: change_this_password
      POSTGRES_DB: trams_noti_db
    ports:
      - "5433:5432"
    volumes:
      - noti_db_data:/var/lib/postgresql/data

volumes:
  noti_db_data:
```

Change the password, then start it from that folder:

```
docker compose up -d
```

The database now listens on your PC at port `5433`. Remember the user, password, port and database name: Part C needs them.

B2. Create the two tables

In the same folder create `schema.sql`:

```
CREATE TABLE IF NOT EXISTS device_tokens (
  id          SERIAL PRIMARY KEY,
  username    TEXT NOT NULL,
  fcm_token   TEXT NOT NULL UNIQUE,
  platform    TEXT NOT NULL DEFAULT 'android',
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_device_tokens_username ON device_tokens
(username);

CREATE TABLE IF NOT EXISTS notifications (
  id            SERIAL PRIMARY KEY,
  to_username   TEXT NOT NULL,
  from_username TEXT,
  title         TEXT NOT NULL,
  body          TEXT NOT NULL DEFAULT '',
  is_read       BOOLEAN NOT NULL DEFAULT false,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_notifications_user ON notifications
(to_username, is_read);
```

Run it:

```
Get-Content schema.sql | docker compose exec -T noti-db psql -U noti_user -d
trams_noti_db
```

You should see `CREATE TABLE` twice. If you use a database tool such as DBeaver or the VS Code database extension instead, connect to `localhost`, port `5433`, and paste the SQL into a query window.

B3. Check it worked

Run these two queries in your database tool. Both should return zero rows and no error:

```
SELECT * FROM device_tokens;
SELECT * FROM notifications;
```

What the tables hold

Table	One row means	Key rule
`device_tokens`	One phone (one app install) that is logged in as a user	`fcm_token` is `UNIQUE`, so a token belongs to only one user at a time. If another user logs in on the same phone, the row moves to that user.
`notifications`	One notification sent to one user	`is_read` starts `false`. The bell count and the notifications list will read from this table.

We use `username` (the value the app already gets as `mUsername` at login) to identify users, because the app has no other user id.

Part C: The notification server

The app cannot send pushes itself, because sending needs a secret key from your Firebase project. A small Express server holds that key, remembers which phone belongs to which user, and asks Firebase to deliver. It lives in the `fcm_server` folder from Part B, outside your Flutter project. Every file below uses ES module style (`import`, not `require`).

C1. Get the Firebase secret key

1. In the Firebase console, open the `trams-test` project, then **Settings** (gear icon) and **Project settings**.
2. Open the **Service accounts** tab and click **Generate new private key**.
3. Save the downloaded file as `serviceAccountKey.json` inside `fcm_server`.

This file is a secret. Never put it in the Flutter app, share it, or upload it to GitHub.

C2. Create the Node project

Run these inside `fcm_server`:

```
npm init -y
npm install express firebase-admin cors pg dotenv
```

Then open the new `package.json` and add one line inside the top-level braces, so Node understands `import`:

```
"type": "module",
```

C3. Secrets file and .gitignore

Create `.env` in `fcm_server`. Use the user, password, port and database name from Part B:

```
DATABASE_URL=postgres://noti_user:change_this_password@localhost:5433/trams_noti_db
```

If your password contains characters such as `@` or `#`, they must be URL-encoded in this line. Easiest is to use a password with only letters and numbers.

Create `.gitignore` in the same folder, so secrets never reach GitHub:

```
.env
serviceAccountKey.json
node_modules
```

C4. db.js (the database connection)

```
// One shared connection pool for the whole server.
import 'dotenv/config';
import pg from 'pg';

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });

export default pool;
```

C5. auth.js (who is calling?)

Every endpoint that needs to know the user uses this one function. For now it reads the username from a request header. This is a **test stand-in**: in production, this is the only place you change, to verify the real access token from your login API and set `req.user` from it. Everything else in the server uses `req.user.username`.

```
// TEST STAND-IN for authentication.
// Production: verify the real access token here and set req.user from it.
// Everything else in the server just uses req.user.username.
export function requireUser(req, res, next) {
  const username = req.get('X-Test-User');
  if (!username) {
    return res.status(401).json({ ok: false, error: 'not logged in' });
  }
  req.user = { username };
  next();
}
```

C6. server.js (the whole file)

```
// Notification server: stores device tokens and notifications,
// and asks Firebase (FCM) to deliver pushes.
// Needs: serviceAccountKey.json (from Firebase console) in the same folder.

import express from 'express';
import cors from 'cors';
import { readFileSync } from 'fs';
import { initializeApp, cert } from 'firebase-admin/app';
import { getMessaging } from 'firebase-admin/messaging';
import pool from './db.js';
import { requireUser } from './auth.js';

// ES modules can't `require` a .json file, so read and parse it instead.
const serviceAccount = JSON.parse(
  readFileSync(new URL('./serviceAccountKey.json', import.meta.url), 'utf8')
);

initializeApp({
  credential: cert(serviceAccount),
});

const app = express();
app.use(cors());
app.use(express.json());

// Quick check that the server is up: open http://localhost:4000 in a
browser.
app.get('/', (req, res) => {
```

```
  res.send('FCM server is running');
});

// Checks the server can talk to the database.
app.get('/health', async (req, res) => {
  try {
    const r = await pool.query('SELECT COUNT(*) FROM device_tokens');
    res.json({ ok: true, deviceTokens: Number(r.rows[0].count) });
  } catch (err) {
    console.error('DB check failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /send-test  (simple test, no database, no login)
// Body (all optional):
//   { "token": "...", "topic": "all_users", "title": "...", "body": "..." }
// If "token" is given, sends to that one phone. Otherwise sends to the
topic.
app.post('/send-test', async (req, res) => {
  const {
    token,
    topic = 'all_users',
    title = 'Test notification',
    body = 'Hello from the Express server',
  } = req.body || {};

  const message = {
    notification: { title, body },
    // high priority = Android delivers right away instead of waiting
    android: { priority: 'high' },
    ...(token ? { token } : { topic }),
  };

  try {
    const id = await getMessaging().send(message);
    console.log('Sent:', id, token ? '(to token)' : `(to topic ${topic})`);
    res.json({ ok: true, id });
  } catch (err) {
    console.error('Send failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});
```

```
// POST /devices   body: { "token": "...", "platform": "android" }
// Called by the app at login. Links this phone's token to the logged-in
user.
// If the token already exists (same phone, different user), it moves to this
user.
app.post('/devices', requireUser, async (req, res) => {
  const { token, platform = 'android' } = req.body || {};
  if (!token) {
    return res.status(400).json({ ok: false, error: 'token required' });
  }
  try {
    await pool.query(
      `INSERT INTO device_tokens (username, fcm_token, platform)
       VALUES ($1, $2, $3)
       ON CONFLICT (fcm_token)
       DO UPDATE SET username = EXCLUDED.username,
                     platform = EXCLUDED.platform,
                     updated_at = now()`,
      [req.user.username, token, platform]
    );
    console.log('Registered device for', req.user.username);
    res.json({ ok: true });
  } catch (err) {
    console.error('Register failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// DELETE /devices   body: { "token": "..." }
// Called by the app at logout. Only removes the link if it belongs to this
user.
app.delete('/devices', requireUser, async (req, res) => {
  const { token } = req.body || {};
  if (!token) {
    return res.status(400).json({ ok: false, error: 'token required' });
  }
  try {
    const r = await pool.query(
      'DELETE FROM device_tokens WHERE fcm_token = $1 AND username = $2',
      [token, req.user.username]
    );
    console.log('Unregistered device for', req.user.username, '- rows:',
```

```
r.rowCount);
    res.json({ ok: true, deleted: r.rowCount });
  } catch (err) {
    console.error('Unregister failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /notifications   body: { "toUsername": "...", "title": "...", "body":
"..." }
// The sender is whoever is logged in (req.user), never taken from the body.
app.post('/notifications', requireUser, async (req, res) => {
  const { toUsername, title, body = '' } = req.body || {};
  if (!toUsername || !title) {
    return res.status(400).json({ ok: false, error: 'toUsername and title
required' });
  }

  try {
    // 1. Save it, so the bell count and notifications page can show it
later.
    const saved = await pool.query(
      `INSERT INTO notifications (to_username, from_username, title, body)
       VALUES ($1, $2, $3, $4)
       RETURNING id`,
      [toUsername, req.user.username, title, body]
    );
    const notificationId = saved.rows[0].id;

    // 2. Find every phone this user is logged in on.
    const devices = await pool.query(
      'SELECT fcm_token FROM device_tokens WHERE username = $1',
      [toUsername]
    );
    const tokens = devices.rows.map((r) => r.fcm_token);

    // 3. Send to all of them.
    let sent = 0;
    let removed = 0;
    if (tokens.length > 0) {
      const result = await getMessaging().sendEachForMulticast({
        tokens,
        notification: { title, body },
```

```
        android: { priority: 'high' },
        data: { notificationId: String(notificationId) },
      });
      sent = result.successCount;

      // 4. Clean up tokens Firebase says are dead (e.g. app was
reinstalled).
      const dead = [];
      result.responses.forEach((r, i) => {
        const code = r.error?.code;
        if (
          !r.success &&
          (code === 'messaging/registration-token-not-registered' ||
            code === 'messaging/invalid-registration-token')
        ) {
          dead.push(tokens[i]);
        }
      });
      if (dead.length > 0) {
        await pool.query(
          'DELETE FROM device_tokens WHERE fcm_token = ANY($1)',
          [dead]
        );
        removed = dead.length;
      }
    }

    console.log(
      `Notification ${notificationId}: ${req.user.username} -> ${toUsername}
(devices ${tokens.length}, sent ${sent}, removed ${removed})`
    );
    res.json({ ok: true, id: notificationId, devices: tokens.length, sent,
removed });
  } catch (err) {
    console.error('Notify failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

const PORT = 4000;
```

```
// '0.0.0.0' lets your phone reach this server over Wi-Fi, not just this PC.
app.listen(PORT, '0.0.0.0', () => {
  console.log(`FCM server listening on port ${PORT}`);
});
```

C7. Run it and test without the app

Start the server. If Windows shows a firewall popup, allow it on **Private networks**.

```
node server.js
```

It prints `FCM server listening on port 4000`. Then check each piece from a second PowerShell window.

1. Database link. Open `http://localhost:4000/health` in a browser. Expected: `{"ok":true,"deviceTokens":0}`.

2. Two accounts on one phone. Pretend Ali then Siti log in on the same phone (same token):

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "ali" } -ContentType "application/json" -Body
'{"token":"fake-token-123"}'
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "siti" } -ContentType "application/json" -Body
'{"token":"fake-token-123"}'
```

Run `SELECT username, fcm_token FROM device_tokens;`. Expected: **one** row, `siti`. The token moved from Ali to Siti.

3. Only the owner can unregister. Ali tries to delete Siti's link:

```
Invoke-RestMethod -Method Delete -Uri http://localhost:4000/devices -Headers
@{ "X-Test-User" = "ali" } -ContentType "application/json" -Body
'{"token":"fake-token-123"}'
```

Expected: `deleted 0`. Repeat with `"siti"` and you get `deleted 1`.

4. A notification for a user with no phone.

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/notifications -
Headers @{ "X-Test-User" = "tester" } -ContentType "application/json" -Body
'{"toUsername":"nobody","title":"Hello","body":"From tester"}'
```

Expected: `devices 0, sent 0`, but the row is saved in `notifications`. Delivery to a real phone is tested in Part E.

Part D: Changes inside lib

This part edits your original `lib` folder. Do steps 1 to 8 in order. `flutter analyze` will show errors until all eight are done, because the files refer to each other. Run it after step 8.

Every "Before" block below is copied from the original `lib`. If your file does not match, stop and compare before editing.

Step 1. New file `lib/constant/fcm_config.dart`

This holds the address of your notification server. Create the file with:

```
// Address of the notification server (fcm_server).
// Run `ipconfig` on your PC and use its IPv4 address. Phone and PC must be
on the same Wi-Fi.
// Keep the :4000 at the end.
const String fcmServerUrl = 'http://192.168.1.23:4000';
```

Change `192.168.1.23` to your PC's IPv4 address. It can change when you reconnect to Wi-Fi, so check it again if the app suddenly cannot reach the server.

Step 2. New file `lib/noti_services/push_service.dart`

Create the folder `lib/noti_services/` and put this file in it. Every call the app makes to the notification server goes through this one file. The folder name matters: the imports in later steps must match it exactly, or you get "Target of URI doesn't exist".

```
import 'package:dio/dio.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/foundation.dart';
import 'package:tram/constant/fcm_config.dart';

class PushService {
  static final Dio _dio = Dio(BaseOptions(
    baseUrl: fcmServerUrl,
    connectTimeout: const Duration(seconds: 5),
    receiveTimeout: const Duration(seconds: 5),
  ));
```

```
  // TEST STAND-IN. Matches requireUser() in the server's auth.js.
  // Production: send the real access token here, e.g. {'Authorization':
'Bearer $accessToken'}.
  static Map<String, String> _authHeaders(String username) =>
      {'X-Test-User': username};

  /// Link this phone's FCM token to the logged-in user.
  static Future<void> registerDevice(String username) async {
    if (kIsWeb) return;
    try {
      final token = await FirebaseMessaging.instance.getToken();
      if (token == null) return;
      await _dio.post(
        '/devices',
        data: {'token': token, 'platform': 'android'},
        options: Options(headers: _authHeaders(username)),
      );
      debugPrint('Push: device registered for $username');
    } catch (e) {
      debugPrint('Push: register failed: $e');
    }
  }

  /// Remove the link between this phone and the user (call before logging
out).
  static Future<void> unregisterDevice(String username) async {
    if (kIsWeb) return;
    try {
      final token = await FirebaseMessaging.instance.getToken();
      if (token == null) return;
      await _dio.delete(
        '/devices',
        data: {'token': token},
        options: Options(headers: _authHeaders(username)),
      );
      debugPrint('Push: device unregistered for $username');
    } catch (e) {
      debugPrint('Push: unregister failed: $e');
    }
  }

  /// If Firebase gives this phone a new token, register the new one.
```

```
  /// getUsername returns the logged-in username, or null when logged out.
  static void listenTokenRefresh(String? Function() getUsername) {
    if (kIsWeb) return;
    FirebaseMessaging.instance.onTokenRefresh.listen((_) {
      final username = getUsername();
      if (username != null) registerDevice(username);
    });
  }

  /// Ask the server to send a notification to another user.
  /// Returns a short message to show on screen.
  static Future<String> sendNotification({
    required String fromUsername,
    required String toUsername,
    required String title,
    required String body,
  }) async {
    try {
      final r = await _dio.post(
        '/notifications',
        data: {'toUsername': toUsername, 'title': title, 'body': body},
        options: Options(headers: _authHeaders(fromUsername)),
      );
      final devices = r.data['devices'];
      return devices == 0
          ? 'Saved, but $toUsername has no phone logged in right now'
          : 'Sent to $toUsername';
    } on DioException catch (e) {
      final serverError =
          e.response?.data is Map ? e.response?.data['error'] : null;
      return 'Failed: ${serverError ?? e.message}';
    }
  }
}
```

Step 3. `lib/main.dart`: start Firebase

3a. Imports. Add two imports at the very top of the file, above the existing `flutter_neumorphic` import:

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart' show kIsWeb;
```

Then add two imports right after the existing `import 'provider/api_provider.dart';` line:

```
import 'firebase_options.dart';
import 'package:tram/noti_services/push_service.dart';
```

3b. Replace `main()`.

Before:

```
void main() async {
  await GetStorage.init();
  await GetStorage.init("Auth");
  runApp(MyApp());
}
```

After:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // We only configured Firebase for Android, so skip it on Chrome/web.
  if (!kIsWeb) {
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
    );
  }
  await GetStorage.init();
  await GetStorage.init("Auth");
  // If Firebase changes this phone's token, tell the server again.
  PushService.listenTokenRefresh(() => GetStorage("Auth").read("name"));
  runApp(MyApp());
}
```

The `kIsWeb` check matters: `firebase_options.dart` only has Android settings, so without it the app would crash on startup in Chrome.

Step 4. `lib/controller/auth/auth_controller.dart`: connect the phone at login, disconnect at logout

4a. Imports. Add these two lines after the existing `import 'package:tram/model/basic/auth_model.dart';` line:

```
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:tram/noti_services/push_service.dart';
```

(`kIsWeb` already comes from the existing `package:flutter/foundation.dart` import.)

4b. In `login()`, add the `if (!kIsWeb)` block between `await box.save();` and the `debugPrint(...)` line.

Before:

```
    await box.save();
    debugPrint('Login - userType: "${data.userProfileData!.mUserType}"');
  }
```

After:

```
    await box.save();
    if (!kIsWeb) {
      try {
        await FirebaseMessaging.instance.subscribeToTopic('all_users');
      } catch (e) {
        debugPrint('FCM subscribe failed: $e');
      }
      // Link this phone's token to the user who just logged in.
      await PushService.registerDevice(data.userProfileData!.mUsername);
    }
    debugPrint('Login - userType: "${data.userProfileData!.mUserType}"');
  }
```

4c. In `logout()`, add a block between `SmartDialog.showLoading(...)` and `await box.erase();`. It must run **before** the erase, because erasing clears the username.

Before:

```
    SmartDialog.showLoading(
      clickMaskDismiss: false,
    );
    await box.erase();
```

After:

```
    SmartDialog.showLoading(
      clickMaskDismiss: false,
    );
    if (!kIsWeb) {
      try {
        await FirebaseMessaging.instance.unsubscribeFromTopic('all_users');
      } catch (e) {
        debugPrint('FCM unsubscribe failed: $e');
      }
      // Remove the link between this phone and the user.
      final username = accessName();
      if (username != null) {
        await PushService.unregisterDevice(username);
      }
    }
    await box.erase();
```

The `try/catch` blocks and the 5-second timeouts in `PushService` stop a server problem from blocking login or logout.

Step 5. `lib/controller/home/root_home_controller.dart`: the bell's count

Add one line right under `RxInt index = 0.obs;`.

Before:

```
class RootHomeController extends GetxController {
  RxInt index = 0.obs;
  final auth = Get.find<AuthController>();
```

After:

```
class RootHomeController extends GetxController {
  RxInt index = 0.obs;
  RxInt notificationCount = 3.obs; // fixed test number for now
  final auth = Get.find<AuthController>();
```

The `3` is a fixed test value so the badge shows. Later it should come from the `notifications` table (count of unread rows).

Step 6. `lib/constant/routes.dart`: register the page

6a. Import. Add this line with the other imports at the top, for example right after `import '../screen/login/login_screen.dart';`:

```
import 'package:tram/screen/notification/notification_screen.dart';
```

6b. Route. Add a route at the very bottom of the `routes` list, just before the closing `];`. After the commented-out blocks is fine.

Before:

```
  // GetPage(
  //   name: '/all_gis_map',
  //   page: () => const AllGisMapScreen(),
  //   binding: AllGisMapBinding(),
  // ),
];
```

After:

```
  // GetPage(
  //   name: '/all_gis_map',
  //   page: () => const AllGisMapScreen(),
  //   binding: AllGisMapBinding(),
  // ),

  GetPage(
    name: '/notifications',
    page: () => const NotificationScreen(),
  ),
];
```

The page has no controller, so it needs no binding.

Step 7. New file `lib/screen/notification/notification_screen.dart`

Create the folder `lib/screen/notification/` and put this file in it. It has the "Send to username" box and button, plus two test buttons from earlier (ask for permission, show this phone's token). The send box only appears on the Android app, not on web.

```
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter/services.dart';
import 'package:flutter_neumorphic/flutter_neumorphic.dart';
import 'package:get/get.dart';

import '../../components/Neumorphic/navbar.dart';

import 'package:tram/controller/auth/auth_controller.dart';
import 'package:tram/noti_services/push_service.dart';

/// Notifications page.
/// It has a box to send a notification to another user, plus test buttons
/// to ask permission and show this phone's FCM token.
/// The real list of notifications will go in the body later.
class NotificationScreen extends StatefulWidget {
  const NotificationScreen({super.key});

  @override
  State<NotificationScreen> createState() => _NotificationScreenState();
}

class _NotificationScreenState extends State<NotificationScreen> {
  String _status = 'Permission: not asked yet';
  String? _token;

  final _toController = TextEditingController();
  String _sendResult = '';
  bool _sending = false;

  Future<void> _sendToUser() async {
    final to = _toController.text.trim();
    final from = Get.find<AuthController>().accessName();
    if (to.isEmpty || from == null) return;

    setState(() {
      _sending = true;
      _sendResult = '';
    });
    final result = await PushService.sendNotification(
      fromUsername: from,
      toUsername: to,
      title: 'New message',
```

```
      body: '$from sent you a notification',
    );
    if (!mounted) return;
    setState(() {
      _sending = false;
      _sendResult = result;
    });
  }

  @override
  void dispose() {
    _toController.dispose();
    super.dispose();
  }

  Future<void> _requestPermission() async {
    final settings = await FirebaseMessaging.instance.requestPermission();
    setState(() {
      _status = 'Permission: ${settings.authorizationStatus.name}';
    });
  }

  Future<void> _getToken() async {
    final token = await FirebaseMessaging.instance.getToken();
    setState(() => _token = token);
  }

  Future<void> _copyToken() async {
    if (_token == null) return;
    await Clipboard.setData(ClipboardData(text: _token!));
    Get.snackbar(
      'Copied',
      'FCM token copied to clipboard',
      colorText: Colors.black,
      margin: const EdgeInsets.all(10),
      duration: const Duration(seconds: 2),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: const NeumorphicNavBar(
```

```
        title: 'Notifications',
        showDrawer: false, // shows a back arrow instead of the menu icon
        color: Color(0xfff3f8fe),
      ),
      backgroundColor: const Color(0xfff3f8fe),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          children: [
            const Text('No notifications yet'),
            const SizedBox(height: 30),
            if (kIsWeb)
              const Text('FCM test buttons only work on the Android app.')
            else ...[
              // Send a notification to another user
              TextField(
                controller: _toController,
                decoration: const InputDecoration(
                  labelText: 'Send to username',
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 12),
              NeumorphicButton(
                onPressed: _sending ? null : _sendToUser,
                child: Text(_sending ? 'Sending...' : 'Send notification'),
              ),
              const SizedBox(height: 8),
              Text(_sendResult),
              const SizedBox(height: 30),

              // Test buttons
              Text(_status),
              const SizedBox(height: 12),
              NeumorphicButton(
                onPressed: _requestPermission,
                child: const Text('1. Allow notifications'),
              ),
              const SizedBox(height: 20),
              NeumorphicButton(
                onPressed: _getToken,
                child: const Text('2. Get FCM token'),
              ),
```

```
              const SizedBox(height: 12),
              if (_token != null) ...[
                SelectableText(
                  _token!,
                  style: const TextStyle(fontSize: 12),
                ),
                const SizedBox(height: 12),
                NeumorphicButton(
                  onPressed: _copyToken,
                  child: const Text('Copy token'),
                ),
              ],
            ],
          ),
        ),
      );
  }
}
```

Step 8. `lib/screen/home/root_home_screen.dart`: the bell in the header

The header's `actions` list (the right side, between the logo and the logout button) holds only the logout button. Replace it.

Before:

```
          actions: [
            NeumorphicButton(
              onPressed: () {
                controller.logout();
              },
              child: const Icon(Icons.logout, size: 20),
            )
          ],
```

After:

```
          actions: [
            if (controller.index.value == 5)
              NeumorphicButton(
                onPressed: () {
                  Get.toNamed('/notifications');
                },
                child: Badge(
                  isLabelVisible: controller.notificationCount.value > 0,
                  label: Text('${controller.notificationCount.value}'),
                  child: const Icon(Icons.notifications_none, size: 20),
                ),
              ),
            NeumorphicButton(
              onPressed: () {
                controller.logout();
              },
              child: const Icon(Icons.logout, size: 20),
            )
          ],
```

How it works:

- `controller.index.value == 5` is the Form Depot tab, so the bell only shows there. The same file already uses this check for the `edit_note` icon.
- The whole screen is wrapped in `Obx(...)`, so the badge number updates by itself when `notificationCount` changes.
- The badge hides when the count is 0.
- Tapping the bell calls `Get.toNamed('/notifications')`, which opens the page from Step 7 through the route from Step 6.

Now run `flutter analyze` in the app's root folder. All eight steps are done, so it should report no issues.

Part E: Build, install and test

You need two Android devices for the full test: phone A logs in as one user, phone B logs in as another. Both phones and your PC must be on the same Wi-Fi.

E1. Get everything running

1. Start the database if it is not running: `docker compose up -d` in `fcm_server`.

2. Start the server: `node server.js` in `fcm_server`. Leave this window open and watch it during the test.
3. Run `ipconfig`, find your PC's **IPv4 Address**, and make sure `lib/constant/fcm_config.dart` (Part D, step 1) uses it.
4. On each phone, open the browser and go to `http://<your-pc-ip>:4000/health`. You should see `{"ok":true,...}`. If not, the phone cannot reach the server: check the Wi-Fi, the IP address and the Windows firewall (allow Private networks).

E2. Build and install the app

In the app's root folder:

```
flutter analyze
flutter build apk --debug
```

`flutter analyze` should report no issues. The first build with Firebase takes several minutes. Then send `build\app\outputs\flutter-apk\app-debug.apk` to both phones, allow installs from that source, and install it. Installing over an older version keeps the phone's Firebase token and login. Uninstalling first gives the phone a new token.

E3. Test with two devices

1. **Log in.** Phone A logs in as one user (ours: `user.level1`). Phone B logs in as a different user. The server window prints `Registered device for <username>` once per login.
2. **Check the database.** Run `SELECT username, left(fcm_token, 25) AS token_start FROM device_tokens;`. You should see one row per logged-in phone.
3. **Prepare phone B.** Open the **Form Depot** tab. The bell with a badge showing 3 sits between the logo and the logout button. Other tabs show no bell. Tap the bell, tap **1. Allow notifications**, choose Allow. The text should read `Permission: authorized`.
4. **Swipe the app away on phone B.** Android shows nothing for a push while the app is open on screen.
5. **Send from phone A.** Open the bell page, type phone B's exact username (it is case-sensitive) in **Send to username**, and tap **Send notification**.
6. **Check the result.** Phone B buzzes within a few seconds. Phone A says `Sent to <username>`. The server window prints `Notification N: ... (devices 1, sent 1, removed 0)`.

If phone A says `Saved, but ... has no phone logged in right now`, phone B is not registered: log out and in again on phone B.

E4. Test two accounts on one phone

On phone B, log out, then log in as a third user. Run the `SELECT` from E3 step 2 again. The row for that token now shows the new username, and there is still only one row for the phone. Send to the old username: phone B no longer buzzes, because the link was removed at logout.

E5. Test the cleanup of dead tokens (optional)

Register a fake token for a real user, then send to that user:

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "<username>" } -ContentType "application/json" -Body
'{"token":"fake-dead-token"}'
```

The next notification to that user reports `devices 2, sent 1, removed 1`, and the fake row is deleted.

If something goes wrong

Symptom	Likely cause	Fix
`firebase` or `flutterfire` "not recognized"	Terminal opened before install, or the Dart tools folder is not on PATH	Reopen the terminal; add the Pub Cache folder to PATH (A2)
Build fails with "Java heap space"	Gradle's default memory is too small	A6
App crashes on start in Chrome	Firebase started on web without web settings	Keep the `kIsWeb` check in `main()` (Part D, step 3)
"Target of URI doesn't exist: ...push_service.dart"	The file is not in `lib/noti_services/`, or the import path differs from the folder name	Move the file to `lib/noti_services/push_service.dart`; make `main.dart`, `auth_controller.dart` and `notification_screen.dart` import exactly that path
Server: `require is not defined in ES module scope`	A file uses `require`, but `package.json` has `"type": "module"`	Use `import` as in Part C; keep the `.js` ending on local imports

Symptom	Likely cause	Fix
Server: `Cannot read properties of undefined (reading 'cert')`	Old `firebase-admin` import style	Use the imports in C6
`/health` returns an error	`DATABASE_URL` wrong, or the database is not running	Check `.env` against Part B; run `docker compose up -d`
Phone's browser cannot open `http://<pc-ip>:4000/health`	Wrong IP, different Wi-Fi, or firewall	E1 step 4
App says `Failed: ... connection ...`	Same as above, or `usesCleartextTraffic` missing	Fix the address in `fcm_config.dart`; check A7; rebuild
`devices 0` for a user who is logged in	Username typed differently (it is case-sensitive), or the phone has not registered since the server restarted or the row was deleted	Copy the exact username from `device_tokens`; log out and in again
`sent 1` but no notification on the phone	Permission off, app open on screen, or battery/Do Not Disturb settings	Tap **1. Allow notifications**; swipe the app away; set battery use to Unrestricted. A reinstall resets the permission
`removed 1` or `NotRegistered`	The token is dead, usually after reinstalling the app	Log in again on that phone to register its new token

Not done yet

This guide gets you to "user A sends, phone B buzzes". These parts are still missing, in the order we plan to do them:

1. **Bell count and list from the database.** `notificationCount` is a fixed `3`, and the notifications page has no list. The `notifications` table already stores everything needed: add an endpoint for the unread count, one for the list, and one to mark a notification as read.
2. **Pushes while the app is open.** Android shows nothing on its own if the app is on screen. A `FirebaseMessaging.onMessage` listener should refresh the bell and the list, and tapping a notification should open `/notifications`.
3. **Pick the recipient from a list** instead of typing a username, and ask for notification permission at login instead of with the test button.

4. **Real authentication.** `requireUser` in `auth.js` and `_authHeaders` in `push_service.dart` are test stand-ins that trust a username header. In production, replace both with the real access token check.
5. **Production setup.** Use your organisation's own Firebase project (repeat A3 and C1 against it), run the server over `https` on a real domain, remove `usesCleartextTraffic`, and keep `serviceAccountKey.json` and `.env` out of git.
6. **iOS.** Only Android is configured. Any iOS or macOS apps that `flutterfire configure` registered by mistake in the Firebase console are harmless and can be deleted under Project settings.